

Universidad de la Costa (CUC)

Facultad de Ingeniería — Programa de Ingeniería de Sistemas

WorldBlog

Plataforma de Blog de Viajes

Estudiante: Juan Diego Torregroza

Materia: Desarrollo Web

Profesor: Luis Toscano

Actividad: Proyecto Final

2026

Tabla de Contenido

1. Descripción
2. Objetivos
3. Justificación
4. Alcance del Proyecto
5. Arquitectura del Sistema
6. Tecnologías y Herramientas
7. Estructura del Proyecto
8. API REST — Endpoints
9. Modelos de Datos
10. Autenticación y Seguridad
11. Frontend — Arquitectura
12. Despliegue
13. Instalación Local
14. Datos de Prueba
15. Repositorio

WorldBlog — Plataforma de Blog de Viajes

Universidad de la Costa (CUC)

Facultad de Ingeniería — Programa de Ingeniería de Sistemas

Estudiante: Juan Diego Torregroza

Materia: Desarrollo Web

Profesor: Luis Toscano

Actividad: Proyecto Final

Año: 2026

Tabla de Contenido

- 1. Descripción
 - 2. Objetivos
 - 3. Justificación
 - 4. Alcance del Proyecto
 - 5. Arquitectura del Sistema
 - 6. Tecnologías y Herramientas
 - 7. Estructura del Proyecto
 - 8. API REST — Endpoints
 - 9. Modelos de Datos
 - 10. Autenticación y Seguridad
 - 11. Frontend — Arquitectura
 - 12. Despliegue
 - 13. Instalación Local
 - 14. Datos de Prueba
 - 15. Repositorio
-

1. Descripción

1.1 Explicación Breve

WorldBlog es una aplicación web fullstack diseñada para que los usuarios compartan y descubran destinos turísticos alrededor del mundo. La plataforma permite a los usuarios registrados crear publicaciones (posts) sobre lugares que han visitado, incluyendo nombre del destino, ubicación, reseña, calificación e imagen. Otros usuarios pueden interactuar con estas publicaciones mediante comentarios, calificaciones y likes, creando una comunidad colaborativa de viajeros.

La aplicación sigue una arquitectura cliente-servidor con un frontend construido en React y un backend en Python con Flask, utilizando MongoDB Atlas como base de datos NoSQL y Railway para el despliegue en producción.

1.2 Problema que Resuelve

Los viajeros que desean compartir sus experiencias y recomendaciones sobre destinos turísticos dependen de plataformas genéricas de redes sociales que no están optimizadas para este propósito. No existe un espacio dedicado donde los usuarios puedan:

- Publicar reseñas estructuradas de destinos con calificaciones numéricas (1-10).
- Visualizar ubicaciones directamente en Google Maps desde la publicación.
- Interactuar con otros viajeros mediante comentarios con calificación propia.
- Seguir a otros usuarios para mantenerse al tanto de sus nuevas publicaciones.
- Compartir destinos fácilmente mediante redes sociales y código QR.

Esto genera que la información sobre destinos turísticos esté dispersa y sin un formato estandarizado que facilite la toma de decisiones de otros viajeros.

1.3 Solución Propuesta

WorldBlog propone una plataforma web especializada en contenido de viajes que ofrece:

- Sistema de registro y autenticación seguro basado en JWT con cookies HttpOnly.

- Creación de publicaciones con campos estructurados: nombre, ubicación, reseña, calificación (1-10) e imagen.
- Sistema de comentarios con calificación independiente por cada comentario.
- Funcionalidad de likes y sistema de seguidores entre usuarios.
- Integración con Google Maps para visualizar la ubicación de los destinos.
- Compartir publicaciones a través de redes sociales (WhatsApp, Telegram, Facebook, X, Email) y código QR.
- Interfaz moderna y responsiva construida con React y HeroUI.
- Documentación automática del backend con Sphinx.

1.4 Resultados Esperados

- Una plataforma funcional donde los usuarios puedan registrarse, iniciar sesión y gestionar su perfil.
 - Un feed de publicaciones con información visual y estructurada sobre destinos turísticos.
 - Interacción social entre usuarios mediante likes, comentarios y sistema de seguidores.
 - Una experiencia de usuario fluida con validación de formularios, manejo de errores y estados de carga.
 - Un sistema backend robusto con API RESTful documentada mediante Sphinx.
 - Despliegue en producción accesible desde cualquier dispositivo.
-

2. Objetivos

2.1 Objetivo General

Desarrollar una aplicación web fullstack que permita a los usuarios compartir, descubrir y valorar destinos turísticos alrededor del mundo, fomentando una comunidad colaborativa de viajeros mediante funcionalidades sociales e interactivas.

2.2 Objetivos Específicos

1. Implementar un sistema de autenticación y autorización seguro utilizando JWT y cookies HttpOnly para proteger las sesiones de los usuarios.

2. Diseñar e implementar una API RESTful con Flask que gestione las operaciones CRUD de usuarios, publicaciones y comentarios.
 3. Desarrollar una interfaz de usuario moderna y responsiva con React, TailwindCSS y HeroUI que ofrezca una experiencia de navegación intuitiva.
 4. Integrar una base de datos NoSQL (MongoDB Atlas) para el almacenamiento flexible de datos de usuarios, publicaciones y comentarios.
 5. Implementar funcionalidades sociales como sistema de likes, comentarios con calificación y seguimiento entre usuarios.
 6. Desplegar la aplicación en Railway con MongoDB Atlas para garantizar disponibilidad y accesibilidad.
 7. Documentar la API del backend utilizando Sphinx para facilitar el mantenimiento y la colaboración.
-

3. Justificación

El turismo es una de las industrias más importantes a nivel global, y la digitalización ha transformado la forma en que las personas planifican sus viajes. Las reseñas y recomendaciones de otros viajeros son un factor determinante en la toma de decisiones.

WorldBlog se justifica por las siguientes razones:

- **Necesidad del mercado:** Existe una demanda de plataformas especializadas donde los viajeros puedan compartir experiencias de forma estructurada, con calificaciones y ubicaciones precisas.
 - **Aprendizaje técnico:** El proyecto integra tecnologías modernas del desarrollo web (React, Flask, MongoDB, Docker, JWT, Railway), permitiendo aplicar conocimientos adquiridos en la materia de Desarrollo Web.
 - **Arquitectura escalable:** La separación entre frontend y backend, junto con el despliegue en servicios independientes, permite escalar cada componente de forma independiente.
 - **Buenas prácticas:** El proyecto implementa patrones de diseño como Context API para manejo de estado, React Query para caché de datos, validación con Zod, y manejo de errores personalizado tanto en frontend como en backend.
-

4. Alcance del Proyecto

4.1 Requerimientos Funcionales

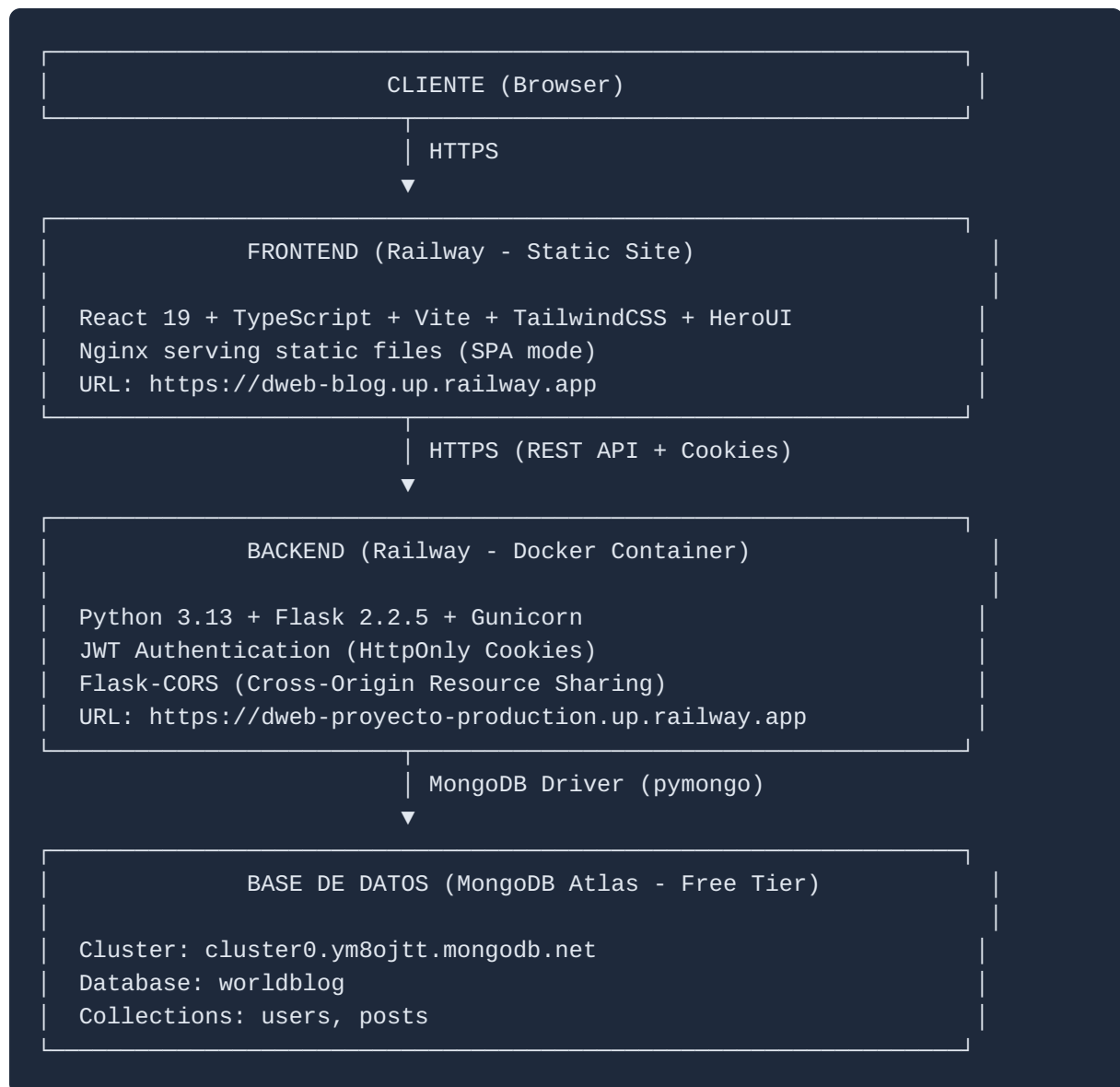
ID	Requerimiento	Descripción
RF-01	Registro de usuarios	Permitir registro con username, email y contraseña con validación de formato
RF-02	Inicio de sesión	Login mediante username o email junto con contraseña
RF-03	Cierre de sesión	Cerrar sesión eliminando la cookie de autenticación
RF-04	Verificación de sesión	Verificar automáticamente si el usuario tiene sesión activa válida
RF-05	Visualización de posts	Mostrar listado de publicaciones con nombre, ubicación, calificación e imagen
RF-06	Detalle de post	Ver detalle completo incluyendo reseña, creador, fecha, likes y comentarios
RF-07	Creación de posts	Crear publicaciones con nombre, ubicación, reseña, calificación (1-10) e imagen URL
RF-08	Edición de posts	Editar únicamente publicaciones propias
RF-09	Eliminación de posts	Eliminar únicamente publicaciones propias
RF-10	Sistema de likes	Dar y quitar like a publicaciones
RF-11	Comentarios	Agregar comentarios con calificación a publicaciones
RF-12	Eliminación de comentarios	Eliminar únicamente comentarios propios
RF-13	Perfil de usuario	Mostrar perfil con publicaciones, seguidores y posts con like
RF-14	Sistema de seguidores	Seguir y dejar de seguir a otros usuarios
RF-15	Compartir publicaciones	Compartir mediante WhatsApp, Telegram, Facebook, X, Email y código QR

ID	Requerimiento	Descripción
RF-16	Integración Google Maps	Visualizar ubicación del destino en Google Maps
RF-17	Navegación sin cuenta	Permitir visualizar el listado de publicaciones sin autenticación

4.2 Requerimientos No Funcionales

ID	Requerimiento	Descripción
RNF-01	Seguridad	Contraseñas cifradas con bcrypt. JWT con cookies HttpOnly y SameSite=None/Secure
RNF-02	Rendimiento	Timeout de 10s en peticiones. Caché con React Query en frontend
RNF-03	Usabilidad	Interfaz responsiva para móvil y escritorio. Estados de carga y mensajes de error claros
RNF-04	Mantenibilidad	Código documentado. Backend con Sphinx. Separación de responsabilidades
RNF-05	Portabilidad	Ejecutable localmente con Docker Compose o desplegable en Railway
RNF-06	Validación	Validación en frontend (Zod + React Hook Form) y backend (validaciones personalizadas)
RNF-07	Disponibilidad	Manejo de errores controlado sin exponer información sensible

5. Arquitectura del Sistema



Flujo de Datos

1. El usuario interactúa con la interfaz React en el navegador.
2. El frontend realiza peticiones HTTP al backend mediante Axios con `withCredentials: true`.
3. El backend procesa la petición, verifica el JWT (si es ruta protegida), ejecuta la lógica de negocio.
4. El backend consulta/modifica MongoDB Atlas a través de PyMongo.
5. La respuesta viaja de vuelta al frontend con datos JSON y cookies actualizadas.

Patrones de Diseño Implementados

- **MVC (Backend):** `api.py` (Controller) → `app.py` (Service/Logic) → `modules/` (Model/Data)
 - **Context API (Frontend):** AuthContext, AppContext, ConfigContext, UIContext, TitleContext
 - **Custom Hooks:** `useApi`, `useAuth`, `useNav`, `useConfig`, `useTitle`, `useToast`
 - **Repository Pattern:** `database.py` encapsula todas las operaciones de MongoDB
 - **Decorator Pattern:** `@verify_token` para proteger endpoints
-

6. Tecnologías y Herramientas

Frontend

Categoría	Herramienta	Versión	Propósito
Framework	React	19.1.0	Biblioteca UI con componentes reactivos
Lenguaje	TypeScript/ JavaScript	5.9.3	Tipado estático y lógica de aplicación
Bundler	Vite	5.4.8	Build tool rápido con HMR
UI Library	HeroUI (NextUI)	2.8.0-beta	Componentes UI pre-diseñados
Estilos	TailwindCSS	4.1.11	Utility-first CSS framework
Routing	React Router DOM	6.27.0	Navegación SPA
Estado servidor	TanStack React Query	5.80.3	Caché y sincronización de datos
Formularios	React Hook Form	7.56.4	Manejo performante de formularios
Validación	Zod	3.25.28	Esquemas de validación type-safe
HTTP Client	Axios	1.9.0	Peticiones HTTP con interceptores
Animaciones	Framer Motion	12.23.0	Animaciones declarativas
QR	qrcode	1.5.4	Generación de códigos QR
Linting	ESLint + Prettier	9.37.0 / 3.6.2	Calidad y formato de código

Backend

Categoría	Herramienta	Versión	Propósito
Framework	Flask	2.2.5	Microframework web Python
WSGI Server	Gunicorn	23.0.0	Servidor de producción
CORS	Flask-CORS	5.0.0	Manejo de Cross-Origin requests
Autenticación	PyJWT	2.10.0	Generación y verificación de JWT
Hashing	bcrypt	4.2.1	Cifrado de contraseñas
Base de datos	PyMongo	4.11.1	Driver MongoDB para Python
Documentación	Sphinx + RTD Theme	8.1.3	Documentación automática
Type Checking	mypy	1.15.0	Verificación estática de tipos
Variables de entorno	python-dotenv	1.0.1	Carga de .env

Infraestructura

Categoría	Herramienta	Propósito
Base de datos	MongoDB Atlas (M0 Free)	Base de datos en la nube
Hosting Backend	Railway (Docker)	Despliegue del contenedor backend
Hosting Frontend	Railway (Docker/Nginx)	Servicio de archivos estáticos
Contenedorización	Docker + Docker Compose	Entorno de desarrollo local
Control de versiones	Git + GitHub	Versionado y colaboración

7. Estructura del Proyecto

```

dweb-proyecto/
├── backend/
│   ├── api.py
│   ├── app.py
│   ├── modules/
│   │   ├── __init__.py
│   │   ├── database.py
│   │   ├── posts.py
│   │   ├── users.py
│   │   └── utils/
│   │       ├── __init__.py
│   │       ├── exceptions.py
│   │       ├── generals.py
│   │       ├── log.py
│   │       └── token.py
│   ├── docs/
│   │   ├── conf.py
│   │   ├── index.rst
│   │   ├── _build/html/
│   │   └── *.rst
│   ├── Dockerfile
│   ├── .dockerignore
│   ├── .env.example
│   ├── railway.json
│   └── requirements.txt
│
├── frontend/
│   ├── src/
│   │   ├── main.tsx
│   │   ├── App.tsx
│   │   ├── App.css
│   │   └── api/
│   │       ├── api.js
│   │       ├── auth.js
│   │       ├── posts.js
│   │       ├── users.js
│   │       └── index.js
│   ├── components/
│   │   ├── ui/
│   │   │   ├── Button.jsx
│   │   │   ├── Container.jsx
│   │   │   ├── Divider.jsx
│   │   │   ├── Dropdown.jsx
│   │   │   ├── Form.jsx
│   │   │   ├── Image.jsx
│   │   │   ├── Input.jsx
│   │   │   ├── Loading.jsx
│   │   │   ├── Modal.jsx
│   │   │   ├── Tabs.jsx
│   │   │   └── Text.jsx
│   │   └── compound/
│   │       └── Comments.jsx
│
└──

```

API REST con Flask
 # Definición de endpoints (Controller)
 # Lógica de negocio (Service Layer)

 # Capa de acceso a datos (Repository)
 # Modelo Post + Comment + gestión
 # Modelo User + User_lite + gestión

 # Excepciones personalizadas
 # Utilidades (bcrypt, validaciones, fechas)
 # Sistema de logging
 # Gestión de JWT
 # Documentación Sphinx
 # Configuración Sphinx
 # Índice de documentación
 # HTML generado (servido en /api/docs/)
 # Archivos fuente RST
 # Imagen Docker para producción

 # Variables de entorno de referencia
 # Configuración de Railway
 # Dependencias Python

 # SPA con React

 # Entry point
 # Componente raíz con providers
 # Estilos globales
 # Capa de comunicación con backend
 # Configuración Axios (baseURL, interceptores)
 # Endpoints de autenticación
 # Endpoints de posts y comentarios
 # Endpoints de usuarios

 # Componentes UI reutilizables

 # Componentes compuestos

```

├── Error.jsx
├── Posts.jsx
├── Share.jsx
├── UserCard.jsx
├── globals/ # Componentes globales
│   ├── Background.jsx
│   └── Menu.jsx
├── layout/
│   └── BackgroundContainer.jsx
├── KindsManager.jsx
├── index.js
├── context/ # Context API (estado global)
│   ├── AppContext.jsx
│   ├── AuthContext.jsx
│   ├── ConfigContext.jsx
│   ├── QueryContext.jsx
│   ├── TitleContext.jsx
│   └── UIContext.jsx
├── hooks/ # Custom Hooks
│   ├── useApi.js
│   ├── useAuth.js
│   ├── useConfig.js
│   ├── useNav.js
│   ├── useTitle.js
│   ├── useToast.js
│   └── index.js
├── pages/ # Páginas/Vistas
│   ├── Welcome.jsx # Landing page (login/register)
│   ├── Blog.jsx # Feed de publicaciones
│   ├── Post.jsx # Detalle de publicación
│   ├── Dashboard.jsx # Crear nueva publicación
│   ├── EditPost.jsx # Editar publicación
│   ├── User.jsx # Perfil de usuario
│   ├── ConfigPage.jsx # Configuración
│   └── index.js
├── routes/ # Configuración de rutas
│   ├── AppRoutes.jsx
│   └── routes.jsx
├── schema/ # Esquemas de validación Zod
│   ├── postSchema.js
│   ├── registerSchema.js
│   └── index.js
├── utils/ # Utilidades
│   ├── compactNumber.ts
│   ├── handleGoogleMaps.ts
│   ├── localStorage.ts
│   ├── qr.ts
│   ├── ratingStars.ts
│   ├── renderMap.jsx
│   ├── timeSince.ts
│   └── index.js
├── assets/ # Recursos estáticos
│   └── background/ # Imágenes de fondo

```



```

├── icons/                # Iconos SVG (redes sociales)
├── Dockerfile            # Build multi-stage con Nginx
├── nginx.conf            # Configuración Nginx para SPA
├── package.json
├── package-lock.json
├── vite.config.js
├── tailwind.config.js
├── tsconfig.json
├── docker-compose.yml    # Orquestación local (dev)
├── seed.py              # Script de datos de prueba
├── .gitignore
└── README.md            # Este archivo

```

8. API REST — Endpoints

Base URL: `https://dweb-proyecto-production.up.railway.app/api`

Autenticación

Método	Endpoint	Descripción	Auth
POST	<code>/app/register</code>	Registrar nuevo usuario	No
POST	<code>/app/login</code>	Iniciar sesión	No
POST	<code>/app/logout</code>	Cerrar sesión	No
GET	<code>/app/verify</code>	Verificar token activo	No

Usuarios

Método	Endpoint	Descripción	Auth
GET	<code>/user/:username</code>	Obtener perfil de usuario	Sí
PUT	<code>/user/:username/follownt</code>	Seguir/dejar de seguir usuario	Sí

Posts

Método	Endpoint	Descripción	Auth
GET	/posts	Obtener todos los posts	No
GET	/post/:id	Obtener detalle de un post	Sí
POST	/post/create	Crear nuevo post	Sí
PUT	/post/:id/edit	Editar un post propio	Sí
DELETE	/post/:id/delete	Eliminar un post propio	Sí
PUT	/post/:id/likent	Dar/quitar like a un post	Sí

Comentarios

Método	Endpoint	Descripción	Auth
PUT	/post/:id/comment	Agregar comentario a un post	Sí
DELETE	/post/:post_id/ comment/:comment_id/ delete	Eliminar comentario propio	Sí

Documentación

Método	Endpoint	Descripción	Auth
GET	/docs/	Documentación Sphinx (HTML)	No

Ejemplos de Request/Response

Registro:

```
// POST /app/register
// Request Body:
{
  "username": "NuevoUsuario",
  "email": "usuario@email.com",
  "password": "MiPassword123"
}
// Response: 201 + Cookie "token" (HttpOnly)
{ "data": "Nuevousuario" }
```

Login:

```
// POST /app/login
// Request Body:
{
  "username_or_email": "usuario@email.com",
  "password": "MiPassword123"
}
// Response: 200 + Cookie "token" (HttpOnly)
{ "data": "Nuevousuario" }
```

Crear Post:

```
// POST /post/create (requiere cookie token)
// Request Body:
{
  "name": "Torre Eiffel",
  "location": "París, Francia",
  "review": "Un lugar increíble...",
  "rating": 9,
  "imageUrl": "https://ejemplo.com/imagen.jpg"
}
// Response: 200
{ "data": "post_id_string" }
```

Agregar Comentario:

```
// PUT /post/:id/comment (requiere cookie token)
// Request Body:
{
  "content": "¡Excelente destino!",
  "rating": 8
}
// Response: 200
{ "data": true }
```

Códigos de Error

Código	Significado
200	Operación exitosa
201	Recurso creado exitosamente
400	Datos inválidos (credenciales, formato)
401	No autorizado (token inválido o ausente)
403	Prohibido (no es el propietario del recurso)
404	Recurso no encontrado
409	Conflicto (username o email ya en uso)
500	Error interno del servidor

9. Modelos de Datos

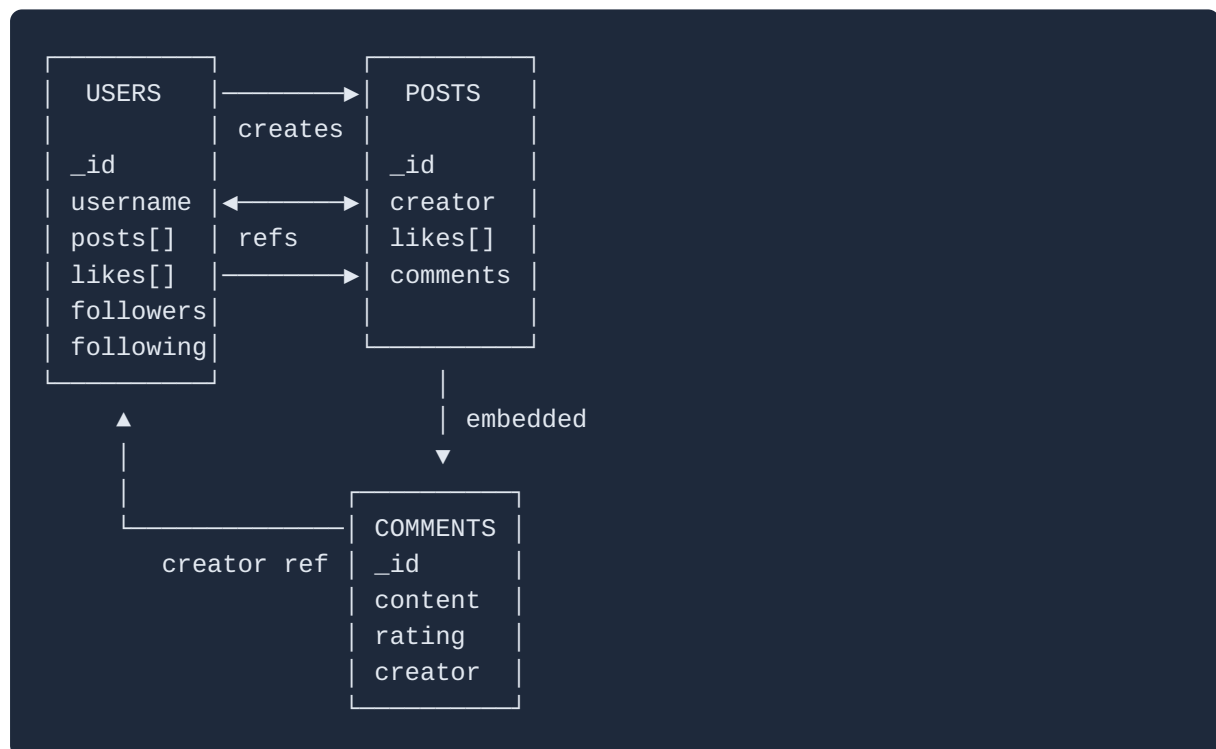
Colección: `users`

```
{
  "_id": "ObjectId",
  "username": "String (único, capitalizado)",
  "email": "String (único, lowercase)",
  "password": "String (hash bcrypt)",
  "posts": ["ObjectId (refs a posts)"],
  "followers": ["ObjectId (refs a users)"],
  "following": ["ObjectId (refs a users)"],
  "likes": ["ObjectId (refs a posts)"],
  "createdAt": "String (ISO 8601)"
}
```

Colección: posts

```
{
  "_id": "ObjectId",
  "name": "String (capitalizado)",
  "location": "String (title case)",
  "review": "String",
  "rating": "Number (1-10)",
  "imageUrl": "String (URL válida de imagen)",
  "creator": {
    "ID": "ObjectId",
    "username": "String"
  },
  "comments": [
    {
      "_id": "ObjectId",
      "content": "String",
      "rating": "Number (1-10)",
      "creator": {
        "ID": "ObjectId",
        "username": "String"
      },
      "createdAt": "String (ISO 8601)"
    }
  ],
  "likes": ["ObjectId (refs a users)"],
  "createdAt": "String (ISO 8601)"
}
```

Diagrama de Relaciones



10. Autenticación y Seguridad

Flujo de Autenticación

1. Usuario envía credenciales (POST `/app/login` o `/app/register`)
2. Backend valida credenciales
3. Backend genera JWT con `{ _id, username }` y exp de 24h
4. JWT se envía como cookie `HttpOnly, Secure, SameSite=None`
5. En cada request protegido:
 - a. Backend lee cookie "token"
 - b. Verifica firma y expiración del JWT
 - c. Extrae datos del usuario y los adjunta a `request.user`
 - d. Ejecuta la lógica del endpoint
 - e. Genera nuevo token (refresh automático) y lo envía como cookie
6. Logout: elimina la cookie

Medidas de Seguridad

Medida	Implementación
Contraseñas	Cifradas con bcrypt (salt rounds: 10)
Tokens	JWT con firma HMAC-SHA256, expiración 24h
Cookies	HttpOnly (no accesible por JS), Secure (solo HTTPS), SameSite=None
CORS	Orígenes permitidos configurables via <code>CORS_ORIGINS</code>
Validación de entrada	Regex para username, email, password en backend
Validación de imágenes	Verificación de Content-Type y magic bytes
Autorización	Verificación de propiedad antes de editar/eliminar
Variables sensibles	Almacenadas como variables de entorno (no en código)

Validaciones de Campos

- **Username:** Letras, números, puntos y guiones bajos. No puede terminar en punto.
 - **Email:** Formato estándar (regex RFC-like).
 - **Password:** Mínimo 8 caracteres, al menos una mayúscula, una minúscula y un número.
 - **Rating:** Número entero entre 1 y 10.
 - **ImageUrl:** URL válida que responda con Content-Type `image/*`.
-

11. Frontend — Arquitectura

Gestión de Estado



Custom Hooks

Hook	Propósito
<code>useAuth</code>	Acceso al contexto de autenticación (user, login, logout, register)
<code>useApi</code>	Operaciones de API (posts, comments, likes, follows) con React Query
<code>useNav</code>	Navegación programática entre páginas
<code>useConfig</code>	Acceso a configuración de la aplicación
<code>useTitle</code>	Gestión dinámica del título de la página (SEO)
<code>useToast</code>	Notificaciones toast para feedback al usuario

Páginas

Página	Ruta	Descripción
Welcome	/	Landing page con formularios de login y registro
Blog	/blog	Feed con todas las publicaciones
Post	/post/:id	Detalle de publicación con comentarios
Dashboard	/dashboard	Formulario para crear nueva publicación
EditPost	/post/:id/edit	Formulario para editar publicación propia
User	/user/:username	Perfil de usuario con sus posts y likes
ConfigPage	/config	Página de configuración

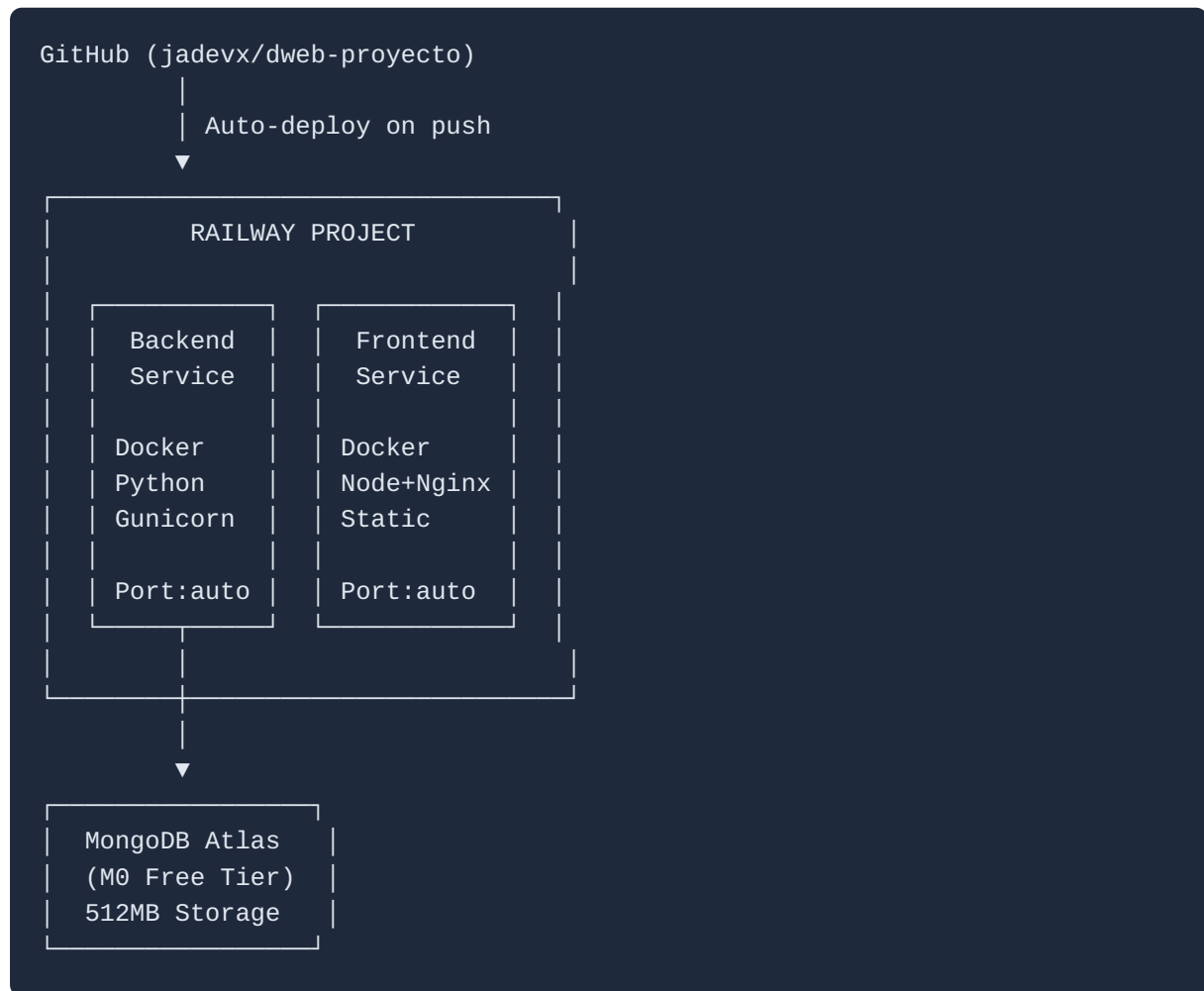
Validación de Formularios

Los formularios utilizan **React Hook Form** para el manejo de estado y **Zod** para la validación:

- `registerSchema.js` — Valida username (3-12 chars), email, password (8+ chars, mayúscula, minúscula, número)
 - `postSchema.js` — Valida nombre, ubicación, reseña, rating (1-10), URL de imagen
-

12. Despliegue

Arquitectura de Despliegue



Variables de Entorno

Backend (Railway):

Variable	Descripción	Ejemplo
MONGO_URI	Connection string de MongoDB Atlas	mongodb+srv:// user:pass@cluster.mongodb.net/ worldblog...
JWT_SECRET_KEY	Clave secreta para firmar JWT	clave_secreta_larga_random
JWT_ALGORITHM	Algoritmo de firma	HS256
JWT_EXPIRATION	Expiración del token en días	1
CORS_ORIGINS	Orígenes permitidos (URL del frontend)	https://dweb- blog.up.railway.app
PORT	Puerto (inyectado por Railway)	Auto

Frontend (Railway):

Variable	Descripción	Ejemplo
VITE_API_URL	URL base de la API del backend	https://dweb-proyecto- production.up.railway.app/ api

13. Instalación Local

Prerrequisitos

- Docker y Docker Compose instalados
- Git

Opción 1: Docker Compose (Recomendado)

```
# Clonar el repositorio
git clone https://github.com/jadevx/dweb-proyecto.git
cd dweb-proyecto

# Crear archivo .env en backend/
cp backend/.env.example backend/.env
# Editar backend/.env con tus valores (para local usar MongoDB local del compose)

# Levantar todos los servicios
docker-compose up --build
```

Esto levanta: - Frontend en `http://localhost:5173` - Backend en `http://localhost:5000` - MongoDB en `localhost:27017`

Opción 2: Manual (sin Docker)

Backend:

```
cd backend
python -m venv venv
source venv/bin/activate # Linux/Mac
# o: venv\Scripts\activate # Windows
pip install -r requirements.txt

# Crear .env con MONGO_URI apuntando a tu MongoDB
cp .env.example .env

# Ejecutar
flask run --host=0.0.0.0
```

Frontend:

```
cd frontend
npm install --legacy-peer-deps

# Crear .env con la URL del backend
echo "VITE_API_URL=http://localhost:5000/api" > .env

npm run dev
```

14. Datos de Prueba

La base de datos está poblada con datos de prueba para facilitar la evaluación:

Usuarios

Username	Email	Contraseña
TalCual	talcual@prof.com	ProfePongame5_:)
M3ssi	m3ssi@test.com	HolaHola123?
W3stcol	w3stcol@test.com	HolaHola123?
F3id	f3id@test.com	HolaHola123?
G0ku	g0ku@test.com	HolaHola123?
S3nk	s3nk@test.com	HolaHola123?

Publicaciones (20 destinos)

Destino	Ubicación	Creador	Rating
Pagoda Chureito	Yamanashi, Japón	M3ssi	9
Pirámides de Egipto	Guiza, Egipto	W3stcol	10
Malecón del Río	Barranquilla, Colombia	W3stcol	8
Islas Lofoten	Nordland, Noruega	F3id	10
Torre Eiffel	París, Francia	S3nk	9
Ciudad Perdida	Santa Marta, Colombia	M3ssi	9
Santuario de Las Lajas	Pasto, Colombia	W3stcol	9
Muralla China	Pekín, China	F3id	10
Salar de Uyuni	Potosí, Bolivia	G0ku	10
Cataratas del Niágara	Ontario, Canadá	S3nk	8
Monte Saint-Michel	Normandía, Francia	G0ku	9
Interlaken	Berna, Suiza	W3stcol	9
Acrópolis de Atenas	Atenas, Grecia	M3ssi	8
Machu Picchu	Cusco, Perú	G0ku	10
Chichén Itzá	Yucatán, México	S3nk	9
Coliseo Romano	Roma, Italia	M3ssi	9
Cristo Redentor	Río de Janeiro, Brasil	S3nk	8
Isla de Pascua	Valparaíso, Chile	F3id	10
Ciudad del Vaticano	Ciudad del Vaticano, Italia	G0ku	9
Stonehenge	Wiltshire, Inglaterra	S3nk	8

Cada publicación tiene entre 2 y 4 comentarios de otros usuarios con calificaciones entre 7 y 10.

15. Repositorio

- **URL:** <https://github.com/jadevx/dweb-proyecto>
- **Rama principal:** `main`

- **Frontend desplegado:** <https://dweb-blog.up.railway.app>
- **Backend desplegado:** <https://dweb-proyecto-production.up.railway.app/api>
- **Documentación API:** <https://dweb-proyecto-production.up.railway.app/api/docs/>